

Ministry of Education and Science of Ukraine
National Technical University of Ukraine “Igor Sikorsky Kyiv Polytechnic
Institute”
Educational and Scientific Institute of Atomic and Thermal Energy
Department of Digital Technologies in Energy (APEPS)

CALCULATION GRAPHICAL WORK
in the discipline “Visualisation of graphical and geometrical information”

Prepared by:
5th year student, IATE
Group TR-42mp
Chervoniak A. B.

Checked by:
Demchyshin A. A.
Kalenyk O. S.

Kyiv – 2024

1 Task description

1.1 Requirements

- Reuse texture mapping from Control task.
- Implement texture scaling (texture coordinates) scaling / rotation around user specified point- odd variants implement scaling, even variants implement rotation
- It has to be possible to move the point along the surface (u,v) space using a keyboard. E.g. keys A and D move the point along u parameter and keys W and S move the point along v parameter.

1.2 Report

Prepare a digital report that holds:

- a title page;
- a chapter describing the task (1 page);
- a chapter describing theory (2 pages);
- a chapter describing the implementation details (2 pages);
- a chapter of user's instruction with screenshots (2 pages);
- a sample of source code (2 pages).

2 Theory

2.1 WebGL Basics

WebGL (Web Graphics Library) is a JavaScript API for rendering interactive 3D graphics within any compatible web browser without the use of plug-ins. WebGL is based on OpenGL ES 2.0 and provides a low-level API for 3D graphics.

2.2 Texture Mapping

Texture mapping is a method for adding detail, surface texture, or color to a 3D model. A texture is an image applied to the surface of a shape or polygon. Texture coordinates (u, v) are used to map the texture to the geometry.

2.3 Texture Rotation

Texture rotation involves rotating the texture coordinates around a specified point. This can be achieved by applying a rotation matrix to the texture coordinates. The rotation matrix for a given angle θ is:

$$R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

2.4 Keyboard Interaction

Keyboard interaction allows users to manipulate the rotation center and angle using keyboard inputs. This can be implemented using event listeners for keydown events in JavaScript.

3 Implementation Details

3.1 HTML Structure

The HTML structure includes elements for displaying the canvas, control panel, and indicators for the rotation center and angle (see Section 5.1 for the code representation)

3.2 JavaScript Implementation

The JavaScript implementation includes functions for initializing WebGL, handling keyboard inputs, and updating the texture coordinates (see Section 5.2 for the code representation)

3.3 SurfaceModel Class

The SurfaceModel class includes methods for rotating texture coordinates, moving the rotation center, and updating the texture coordinate buffer (see Section 5.3 for the code representation)

4 User's Instruction with Screenshots

4.1 Open the HTML File

Open the index.html file in a web browser. You should see a canvas and a control panel. A visual representation is provided in Figure 4.1.1.

A Sievert's Surface Surface

Drag your mouse on the canvas to rotate it. (On a touch screen, you can use your finger.)

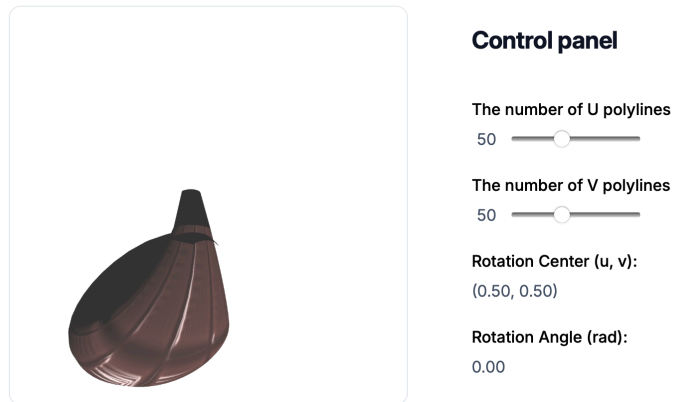


Figure 4.1.1 – Open the HTML File

4.2 Rotate the Texture

Press the R key to rotate the texture. The rotation angle will be updated in the control panel. A visual representation is provided in Figure 4.2.1.

A Sievert's Surface Surface

Drag your mouse on the canvas to rotate it. (On a touch screen, you can use your finger.)

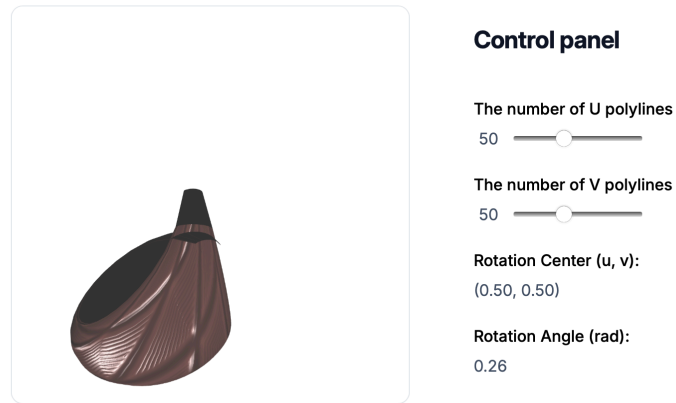


Figure 4.2.1 – Rotate the Texture

4.3 Move the Rotation Center

Use the A, D, W, and S keys to move the rotation center. The center coordinates will be updated in the control panel. A visual representation is provided in Figure 4.3.1.

A Sievert's Surface Surface

Drag your mouse on the canvas to rotate it. (On a touch screen, you can use your finger.)

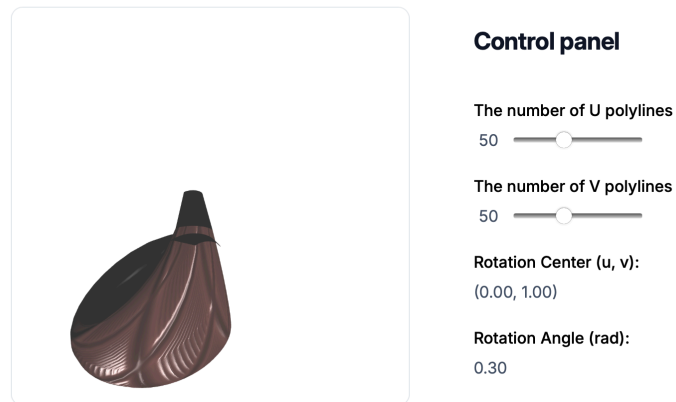


Figure 4.3.1 – Move the Rotation Center

4.4 Adjust the Number of Polylines

Use the sliders in the control panel to adjust the number of U and V polylines. The surface will be updated accordingly. A visual representation is provided in Figure 4.4.1.

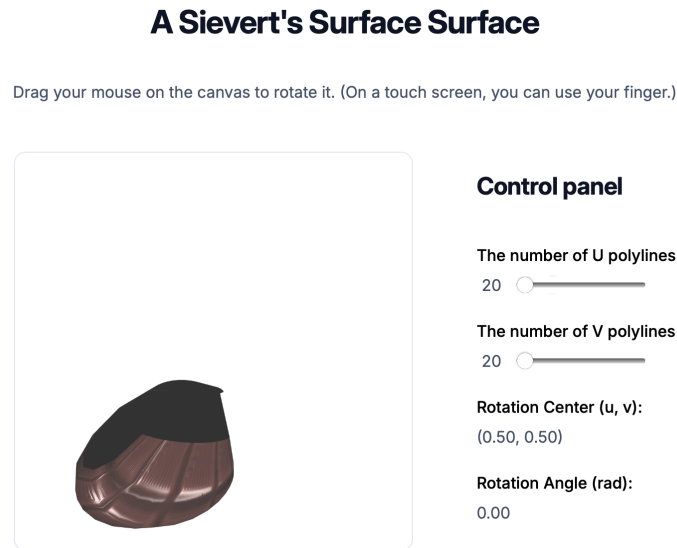


Figure 4.4.1 – Adjust the Number of Polylines

5 Sample of Source Code

5.1 HTML Structure

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>A Sievert's Surface | Andrei Chervoniak</title>
  <link rel="stylesheet" href="./style.css" />
</head>

<body>
  <h1>A Sievert's Surface Surface</h1>
  <p>
    Drag your mouse on the canvas to rotate it. (On a touch screen, you can
```

```

        use your finger.)
    </p>
    <article class="canvas-container">
        <canvas width="1200" height="1200" style="width: 400px; height:
400px"></canvas>
        <section class="canvas-container__controls">
            <h2>Control panel</h2>
            <div class="form-input">
                <label for="u-stepper">The number of U polylines</label>
                <div class="form-input__slider">
                    <span id="u-counter" class="form-input__value">50</span>
                    <input type="range" name="u-stepper" id="u-stepper" min="20"
max="100" value="50">
                </div>
            </div>
            <div class="form-input">
                <label for="v-stepper">The number of V polylines</label>
                <div class="form-input__slider">
                    <span id="v-counter" class="form-input__value">50</span>
                    <input type="range" name="v-stepper" id="v-stepper" min="20"
max="100" value="50">
                </div>
            </div>
            <div class="form-input">
                <label>Rotation Center (u, v):</label>
                <span id="rotation-center">(0.5, 0.5)</span>
            </div>
            <div class="form-input">
                <label>Rotation Angle (rad):</label>
                <span id="rotation-angle">0</span>
            </div>
        </section>
    </article>

    <script src="./Utils/trackball-rotator.js"></script>
    <script src="./Utils/m4.js"></script>
    <script src="./shader.js"></script>
    <script src="./main.js" type="module"></script>
</body>

```

</html>

5.2 JavaScript Implementation

```
import { SurfaceModel } from "./SurfaceModel.js";
import { ShaderProgram } from "./ShaderProgram.js";

let gl;
let surface;
let program;
let ball;

function getUVSteps() {
    return {
        u: Number.parseInt(document.getElementById("u-stepper").value,
10),
        v: Number.parseInt(document.getElementById("v-stepper").value,
10),
    };
}

function initSurface() {
    const { u, v } = getUVSteps();
    surface = new SurfaceModel("Richmond's Minimal Surface", u, v);
    surface.createSurfaceData();
    surface.initBuffer(gl);
    surface.loadTextures(gl);
}

function initShaderProgram() {
    program = new ShaderProgram("Basic");
    program.init(gl, vertexShaderSource, fragmentShaderSource);
    program.use(gl);
}

function setupUIControls() {
    const stepperTypes = ["u", "v"];

    // biome-ignore lint/complexity/noForEach: <explanation>
    stepperTypes.forEach((type) => {
```

```

const stepper = document.getElementById(`${type}-stepper`);
const counter = document.getElementById(`${type}-counter`);

if (!stepper) {
    console.error(`Stepper element with id '${type}-stepper'
not found`);

    return;
}

stepper.addEventListener("input", (e) => {
    if (counter) {
        counter.textContent = e.target.value;
    } else {
        console.warn(`Counter element with id '${type}-
counter' not found`);
    }
    initSurface();
    draw();
});
});
}

function animateLight(time) {
    const radius = 10.0;
    const speed = 0.001;
    const x = radius * Math.cos(time * speed);
    const z = radius * Math.sin(time * speed);
    const y = 5.0;

    if (program) {
        gl.uniform3f(program.lightDirectionUni, x, y, z);
        draw();
    }
    requestAnimationFrame(animateLight);
}

function updateIndicators() {
    const rotationCenter = document.getElementById("rotation-center");
    const rotationAngle = document.getElementById("rotation-angle");

```

```

        rotationCenter.textContent =
`(${surface.rotationCenter.u.toFixed(2)} ,
${surface.rotationCenter.v.toFixed(2)})`;
        rotationAngle.textContent = surface.rotationAngle.toFixed(2);
    }

    function handleKeyDown(event) {
        const step = 0.1;
        switch (event.key) {
            case "a":
            case "A":
                surface.moveRotationCenter(-step, 0);
                break;
            case "d":
            case "D":
                surface.moveRotationCenter(step, 0);
                break;
            case "w":
            case "W":
                surface.moveRotationCenter(0, step);
                break;
            case "s":
            case "S":
                surface.moveRotationCenter(0, -step);
                break;
            case "r":
            case "R":
                surface.rotateTextureCoordinates(surface.rotationAngle +
Math.PI / 180);

                surface.updateTexCoordBuffer(gl);
                break;
        }
        updateIndicators();
        draw();
    }

    function draw() {
        gl.clearColor(1, 1, 1, 1);
        gl.clear(gl.COLOR_BUFFER_BIT | gl.DEPTH_BUFFER_BIT);
    }

```

```

const projection = m4.perspective(Math.PI / 8, 1, 0.1, 100);
const modelView = ball.getViewMatrix();

const rotateToPointZero = m4.axisRotation(
    [Math.SQRT1_2, Math.SQRT1_2, 0],
    0.7,
);
const translateToPointZero = m4.translation(0, 0, -10);

const matAcc0 = m4.multiply(rotateToPointZero, modelView);
const matAcc1 = m4.multiply(translateToPointZero, matAcc0);

const modelViewProjection = m4.multiply(projection, matAcc1);
gl.uniformMatrix4fv(program.matrixUni, false, modelViewProjection);

const normalMatrix = m4.transpose(m4.inverse(matAcc1));
gl.uniformMatrix4fv(program.normalMatrixUni, false, normalMatrix);

gl.uniform3fv(program.viewPositionUni, [0.0, 0.0, 5.0]);
gl.uniform3f(program.ambientColorUni, 0.2, 0.2, 0.2);
gl.uniform3f(program.diffuseColorUni, 0.7, 0.7, 0.7);
gl.uniform3f(program.specularColorUni, 1.0, 1.0, 1.0);
gl.uniform1f(program.shininessUni, 32.0);

surface.bindTextures(gl, program);
surface.draw(gl, program);
}

function init() {
    try {
        const canvas = document.querySelector("canvas");
        gl = canvas.getContext("webgl");
        if (!gl) {
            throw "Browser does not support WebGL";
        }
        ball = new TrackballRotator(canvas, draw, 0);

        initShaderProgram();
        initSurface();
        gl.enable(gl.DEPTH_TEST);
    }
}

```

```

        setupUIControls();
        draw();
        animateLight(0);

        window.addEventListener("keydown", handleKeyDown);
    } catch (e) {
        console.error(`Initialization error: ${e}`);
        const errorMessage = document.createElement("p");
        errorMessage.textContent = `Sorry, could not initialize the
WebGL graphics context: ${e}`;
        document.body.appendChild(errorMessage);
    }
}
document.addEventListener("DOMContentLoaded", init);

```

5.3 SurfaceModel Class

```

export class SurfaceModel {
    constructor(name, uSteps, vSteps) {
        this.name = name;
        this.uSteps = uSteps || 50; // Default to 50 if not provided
        this.vSteps = vSteps || 50; // Default to 50 if not provided
        this.uLines = [];
        this.vLines = [];
        this.vertices = [];
        this.indices = [];
        this.normals = [];
        this.texCoords = [];
        this.tangents = [];
        this.textures = {};
        this.rotationCenter = { u: 0.5, v: 0.5 }; // Default rotation
center
        this.rotationAngle = 0; // Default rotation angle
    }

    createSurfaceData() {
        this.generateUVLines();
        this.generateVertices();
        this.generateIndices();
    }
}

```

```

        this.generateNormals();
        this.generateTexCoords();
        this.generateTangents();
    }

    generateUVLines() {
        const uMin = (-0.95 * Math.PI) / 2;
        const uMax = (0.95 * Math.PI) / 2;
        const vMin = -0.95 * Math.PI;
        const vMax = 0.95 * Math.PI;
        const c = 1;

        const du = (uMax - uMin) / this.uSteps;
        const dv = (vMax - vMin) / this.vSteps;

        // Generate U lines
        for (let uIndex = 0; uIndex <= this.uSteps; uIndex++) {
            const u = uMin + uIndex * du;
            const uLine = [];
            for (let vIndex = 0; vIndex <= this.vSteps; vIndex++) {
                const v = vMin + vIndex * dv;
                const phi =
                    -u / Math.sqrt(c + 1) + Math.atan(Math.sqrt(c
+ 1) * Math.tan(u));

                const a = 2 / (c + 1 - c * Math.sin(v) ** 2 *
Math.cos(u) ** 2);

                const r =
                    (a / Math.sqrt(c)) *
                    Math.sqrt((c + 1) * (1 + c * Math.sin(u) **
2)) *
                    Math.sin(v);
                const point = this.calculateSurfacePoint(u, v, c,
phi, a, r);

                uLine.push(point);
            }
            this.uLines.push(uLine);
        }

        // Generate V lines by transposing the uLines array
        for (let vIndex = 0; vIndex <= this.vSteps; vIndex++) {

```

```

        const vLine = this.uLines.map((uLine) => uLine[vIndex]);
        this.vLines.push(vLine);
    }
}

calculateSurfacePoint(u, v, c, phi, a, r) {
    // Parametric equation for Sievert's Surface
    const x = r * Math.cos(phi);
    const y = r * Math.sin(phi);
    const z = (Math.log(Math.tan(v / 2)) + a * (c + 1) * Math.cos(v))
/ 2;

    return [x, y, z];
}

generateVertices() {
    this.vertices = this.uLines.flat(2);
}

generateIndices() {
    this.indices = [];
    for (let u = 0; u < this.uSteps; u++) {
        for (let v = 0; v < this.vSteps; v++) {
            const topLeft = u * (this.vSteps + 1) + v;
            const topRight = topLeft + 1;
            const bottomLeft = (u + 1) * (this.vSteps + 1) + v;
            const bottomRight = bottomLeft + 1;

            this.indices.push(topLeft, bottomLeft, topRight);
            this.indices.push(topRight,
                                bottomLeft,
bottomRight);
        }
    }
}

generateNormals() {
    const normals = new Array(this.vertices.length).fill(0);

    for (let i = 0; i < this.indices.length; i += 3) {
        const i1 = this.indices[i] * 3;
        const i2 = this.indices[i + 1] * 3;

```

```

        const i3 = this.indices[i + 2] * 3;

        const v1 = this.vertices.slice(i1, i1 + 3);
        const v2 = this.vertices.slice(i2, i2 + 3);
        const v3 = this.vertices.slice(i3, i3 + 3);

        const normal = this.calculateFaceNormal(v1, v2, v3);

        for (let j = 0; j < 3; j++) {
            const idx = this.indices[i + j] * 3;
            normals[idx] += normal[0];
            normals[idx + 1] += normal[1];
            normals[idx + 2] += normal[2];
        }
    }

    this.normals = this.normalizeVectors(normals);
}

calculateFaceNormal(v1, v2, v3) {
    const edge1 = [v2[0] - v1[0], v2[1] - v1[1], v2[2] - v1[2]];
    const edge2 = [v3[0] - v1[0], v3[1] - v1[1], v3[2] - v1[2]];

    return [
        edge1[1] * edge2[2] - edge1[2] * edge2[1],
        edge1[2] * edge2[0] - edge1[0] * edge2[2],
        edge1[0] * edge2[1] - edge1[1] * edge2[0],
    ];
}

generateTexCoords() {
    for (let uIndex = 0; uIndex <= this.uSteps; uIndex++) {
        for (let vIndex = 0; vIndex <= this.vSteps; vIndex++) {
            this.texCoords.push(uIndex / this.uSteps, vIndex /
this.vSteps);
        }
    }
}

generateTangents() {

```

```

        for (let uIndex = 0; uIndex <= this.uSteps; uIndex++) {
            for (let vIndex = 0; vIndex <= this.vSteps; vIndex++) {
                this.tangents.push(1, 0, 0); // Placeholder tangent
vector
            }
        }
    }

    normalizeVectors(vectors) {
        const normalized = [];
        for (let i = 0; i < vectors.length; i += 3) {
            const length = Math.sqrt(
                vectors[i] ** 2 + vectors[i + 1] ** 2 + vectors[i +
2] ** 2,
            );
            if (length > 0) {
                normalized.push(
                    vectors[i] / length,
                    vectors[i + 1] / length,
                    vectors[i + 2] / length,
                );
            } else {
                normalized.push(0, 0, 0);
            }
        }
        return normalized;
    }

    loadTexture(gl, url) {
        const texture = gl.createTexture();
        gl.bindTexture(gl.TEXTURE_2D, texture);

        const level = 0;
        const internalFormat = gl.RGBA;
        const width = 1;
        const height = 1;
        const border = 0;
        const srcFormat = gl.RGBA;
        const srcType = gl.UNSIGNED_BYTE;
        const pixel = new Uint8Array([0, 0, 255, 255]); // blue

```

```

        gl.texImage2D(
            gl.TEXTURE_2D,
            level,
            internalFormat,
            width,
            height,
            border,
            srcFormat,
            srcType,
            pixel,
        );

const image = new Image();
image.onload = () => {
    gl.bindTexture(gl.TEXTURE_2D, texture);
    gl.texImage2D(
        gl.TEXTURE_2D,
        level,
        internalFormat,
        srcFormat,
        srcType,
        image,
    );

    if (isPowerOf2(image.width) && isPowerOf2(image.height))
    {
        gl.generateMipmap(gl.TEXTURE_2D);
    } else {
        gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_WRAP_S,
gl.CLAMP_TO_EDGE);
        gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_WRAP_T,
gl.CLAMP_TO_EDGE);
        gl.texParameteri(gl.TEXTURE_2D,
gl.TEXTURE_MIN_FILTER, gl.LINEAR);
    }
};
image.src = url;

return texture;
}

```

```

loadTextures(gl) {
    this.textures.diffuse = this.loadTexture(
        gl,
        "assets/chocolate_basecolor.png",
    );
    this.textures.specular = this.loadTexture(
        gl,
        "assets/chocolate_height.png",
    );
    this.textures.normal = this.loadTexture(gl,
"assets/chocolate_normal.png");
}

rotateTextureCoordinates(angle) {
    this.rotationAngle = angle;
    const cosAngle = Math.cos(angle);
    const sinAngle = Math.sin(angle);
    const { u: centerU, v: centerV } = this.rotationCenter;

    for (let i = 0; i < this.texCoords.length; i += 2) {
        const u = this.texCoords[i] - centerU;
        const v = this.texCoords[i + 1] - centerV;

        this.texCoords[i] = u * cosAngle - v * sinAngle + centerU;
        this.texCoords[i + 1] = u * sinAngle + v * cosAngle +
centerV;
    }
}

moveRotationCenter(deltaU, deltaV) {
    this.rotationCenter.u += deltaU;
    this.rotationCenter.v += deltaV;
    this.rotationCenter.u = Math.max(0, Math.min(1,
this.rotationCenter.u));
    this.rotationCenter.v = Math.max(0, Math.min(1,
this.rotationCenter.v));
}

updateTexCoordBuffer(gl) {

```

```

        gl.bindBuffer(gl.ARRAY_BUFFER, this.texCoordBuffer);
        gl.bufferData(
            gl.ARRAY_BUFFER,
            new Float32Array(this.texCoords),
            gl.STATIC_DRAW,
        );
    }

    bindTextures(gl, program) {
        gl.activeTexture(gl.TEXTURE0);
        gl.bindTexture(gl.TEXTURE_2D, this.textures.diffuse);
        gl.uniform1i(program.diffuseTextureUni, 0);

        gl.activeTexture(gl.TEXTURE1);
        gl.bindTexture(gl.TEXTURE_2D, this.textures.specular);
        gl.uniform1i(program.specularTextureUni, 1);

        gl.activeTexture(gl.TEXTURE2);
        gl.bindTexture(gl.TEXTURE_2D, this.textures.normal);
        gl.uniform1i(program.normalTextureUni, 2);
    }

    initBuffer(gl) {
        this.vertexBuffer = gl.createBuffer();
        gl.bindBuffer(gl.ARRAY_BUFFER, this.vertexBuffer);
        gl.bufferData(
            gl.ARRAY_BUFFER,
            new Float32Array(this.vertices),
            gl.STATIC_DRAW,
        );

        this.indexBuffer = gl.createBuffer();
        gl.bindBuffer(gl.ELEMENT_ARRAY_BUFFER, this.indexBuffer);
        gl.bufferData(
            gl.ELEMENT_ARRAY_BUFFER,
            new Uint16Array(this.indices),
            gl.STATIC_DRAW,
        );

        this.normalBuffer = gl.createBuffer();
    }

```

```

        gl.bindBuffer(gl.ARRAY_BUFFER, this.normalBuffer);
        gl.bufferData(
            gl.ARRAY_BUFFER,
            new Float32Array(this.normals),
            gl.STATIC_DRAW,
        );

        this.texCoordBuffer = gl.createBuffer();
        gl.bindBuffer(gl.ARRAY_BUFFER, this.texCoordBuffer);
        gl.bufferData(
            gl.ARRAY_BUFFER,
            new Float32Array(this.texCoords),
            gl.STATIC_DRAW,
        );

        this.tangentBuffer = gl.createBuffer();
        gl.bindBuffer(gl.ARRAY_BUFFER, this.tangentBuffer);
        gl.bufferData(
            gl.ARRAY_BUFFER,
            new Float32Array(this.tangents),
            gl.STATIC_DRAW,
        );
    }

    draw(gl, program) {
        gl.bindBuffer(gl.ARRAY_BUFFER, this.vertexBuffer);
        gl.vertexAttribPointer(program.vertexAttrib, 3, gl.FLOAT,
false, 0, 0);

        gl.enableVertexAttribArray(program.vertexAttrib);

        gl.bindBuffer(gl.ARRAY_BUFFER, this.normalBuffer);
        gl.vertexAttribPointer(program.normalAttrib, 3, gl.FLOAT,
false, 0, 0);

        gl.enableVertexAttribArray(program.normalAttrib);

        gl.bindBuffer(gl.ARRAY_BUFFER, this.texCoordBuffer);
        gl.vertexAttribPointer(program.texCoordAttrib, 2, gl.FLOAT,
false, 0, 0);

        gl.enableVertexAttribArray(program.texCoordAttrib);
    }

```

```

        gl.bindBuffer(gl.ARRAY_BUFFER, this.tangentBuffer);
        gl.vertexAttribPointer(program.tangentAttrib, 3, gl.FLOAT,
false, 0, 0);

        gl.enableVertexAttribArray(program.tangentAttrib);

        gl.bindBuffer(gl.ELEMENT_ARRAY_BUFFER, this.indexBuffer);
        gl.drawElements(gl.TRIANGLES, this.indices.length,
gl.UNSIGNED_SHORT, 0);
    }
}

function isPowerOf2(value) {
    return (value & (value - 1)) === 0;
}

```